ROS 2 Mini Project

URDF Modeling and RViz2 Visualization of an Industrial Robotic Arm

Student: Harsh Koche
Project folder: ros2mini project
Platform: Ubuntu 22.04 · ROS 2 Humble · URDF/Xacro · RViz2

My design approach

I began by thinking about the robot as a set of rigid links and the motions that connect them. I used `world` as a stable reference, fixed the base to it, and arranged the arm links in a serial chain. At the end of the arm, I attached two separate sliding finger links to represent the gripper. This structure keeps the arm motion and gripper motion easy to identify in both the Xacro and the TF tree.

> **Why these joint types?** I used revolute joints for the articulated arm because those links rotate about an axis. I used prismatic joints for the gripper fingers because they translate along an axis. I kept the finger joints as separate branches so their direction and travel limits can be specified independently.

The included model contains five revolute arm joints and two prismatic gripper joints. I describe the gripper separately from the arm so the function of each movable joint is clear.

Question 1 — Robot structure design

Question: Define the kinematic structure of the arm and label its joints and links.

Answer: I represented the root reference as `world` and attached the robot base, `base_link`, with the fixed joint `base_joint`. Five revolute joints connect the arm links sequentially. The final arm link, `link_5`, is the parent of the two prismatic gripper fingers.

```
world
└── base_joint (fixed) → base_link
    └── joint_1 → link_1
        └── joint_2 → link_2
            └── joint_3 → link_3
                └── joint_4 → link_4
                    └── joint_5 → link_5
                        ├── joint_6 (prismatic) → link_6
```

```
                    └── joint_7 (prismatic) → link_7
```

The complete model has nine links including the reference frame, one fixed joint, five revolute arm joints, and two prismatic gripper joints.

Question 2 — URDF link definition

Question: Define the links, including visual and collision geometry.

Answer: In my Xacro, `base_link` and `link_1` through `link_7` each have a visual element and a collision element. Their geometry references STL meshes in the package's `meshes/` directory. Visual elements specify material colours for rendering in RViz2; collision elements describe the link shapes for collision-aware tools. The robot links also include inertial properties. The `world` link is a reference frame and does not need geometry.

Xacro source — root frame, base link and mesh geometry

```
--- Robot root and base visual/collision geometry ---
<robot
  name="robot_arm_urdf"
  xmlns:xacro="http://www.ros.org/wiki/xacro">
  <link name="world"/>
  <joint name="base_joint" type="fixed">
    <parent link="world"/>
    <child link="base_link"/>
    <origin rpy="0 0 0" xyz="0.0 0.0 0.17"/>
  </joint>
  <link
    name="base_link">
    <inertial>
      <origin
        xyz="4.1633E-17 0 0.073876"
        rpy="0 0 0" />
      <mass
        value="17.147" />
      <inertia
        ixx="0.192"
        ixy="-4.9423E-10"
        ixz="1.2752E-17"
        iyy="0.192"
        iyz="2.6417E-18"
        izz="0.29544" />
    </inertial>
    <visual>
      <origin
        xyz="0 0 0"
        rpy="0 0 0" />
      <geometry>
        <mesh
          filename="package://robot_arm_urdf/meshes/base_link.STL" />
      </geometry>
      <material
        name="">
        <color
          rgba="0.79216 0.81961 0.93333 1" />
      </material>
    </visual>
    <collision>
      <origin
        xyz="0 0 0"
        rpy="0 0 0" />
      <geometry>
        <mesh
          filename="package://robot_arm_urdf/meshes/base_link.STL" />
      </geometry>
    </collision>
  </link>
```

Figure 1. Xacro excerpt showing the robot root, fixed base, mesh visual, and collision geometry.

Question 3 — Joint definition

Question: Define joint types, parent–child relationships, axes, and limits.

Answer: Each moving joint names its parent and child links, type of motion, axis, and allowed range. Revolute limits are angular values in radians. Prismatic limits are linear distances in metres. The Xacro also provides effort and velocity limits.

Joint	Type	Parent → child	Axis (XYZ)	Position limits	Effort / velocity
base_joint	Fixed	world → base_link	—	—	—
joint_1	Revolute	base_link → link_1	0 1 0	0 to 3.142 rad	300 N·m / 3 rad/s
joint_2	Revolute	link_1 → link_2	0 0 1	−1.57 to 1.57 rad	200 N·m / 3 rad/s
joint_3	Revolute	link_2 → link_3	0 0 1	0 to 3.142 rad	200 N·m / 3 rad/s
joint_4	Revolute	link_3 → link_4	0 0 1	−3.142 to 0 rad	200 N·m / 3 rad/s
joint_5	Revolute	link_4 → link_5	1 0 0	0 to 3.142 rad	200 N·m / 3 rad/s
joint_6	Prismatic	link_5 → link_6	0 0 −1	0 to 0.03 m	100 N / 3 m/s
joint_7	Prismatic	link_5 → link_7	0 0 1	−0.03 to 0 m	100 N / 3 m/s

The first value in the final column is the effort limit (N·m for revolute joints and N for prismatic joints); the second is the velocity limit.

Joint source — arm joints 1 to 3

These Xacro excerpts show how I connect the base and arm links, set each rotation axis, and constrain the joint range.

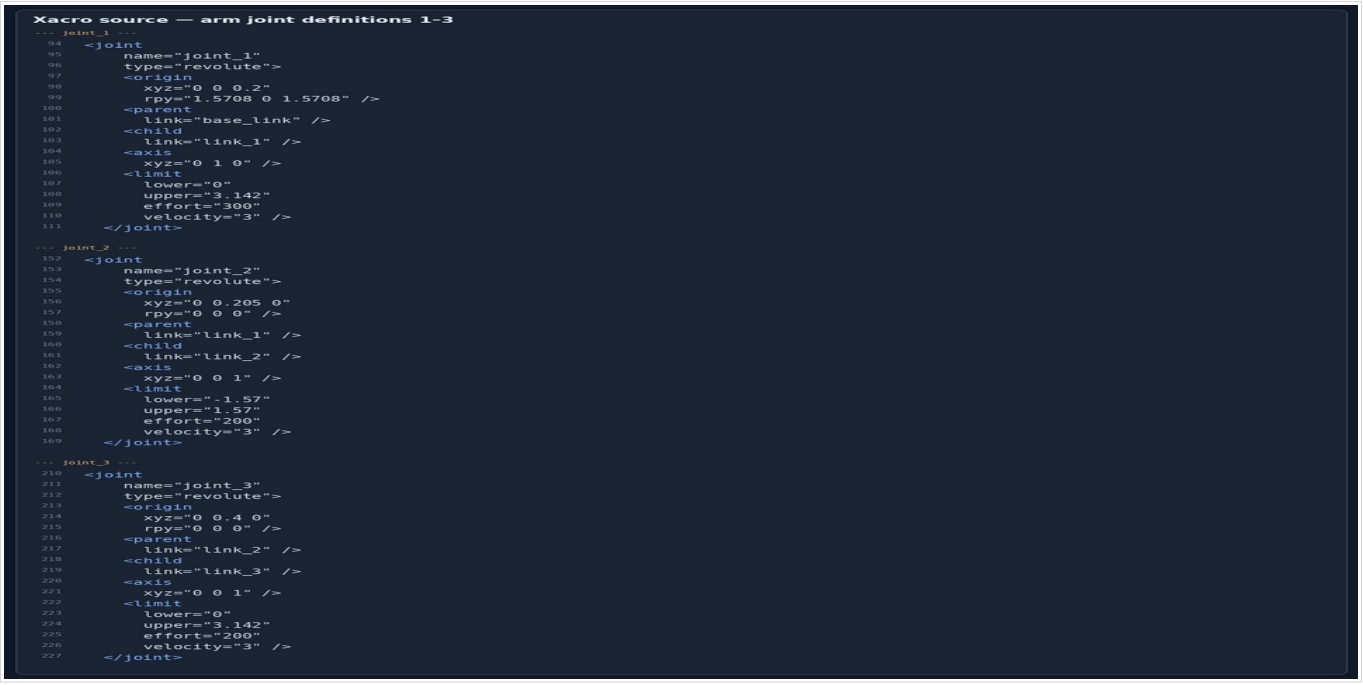


Figure 2. Source excerpts for revolute joints 1–3.

Joint source — arm joints 4 and 5

The next two revolute joints continue the arm chain and define the wrist-side orientation axes.



Figure 3. Source excerpts for revolute joints 4–5.

Joint source — gripper fingers

The gripper uses two prismatic joints attached to `link_5`. The opposite axis directions and 3 cm limits represent the two fingers' sliding motion.



Figure 4. Source excerpts for the prismatic finger joints, their axes, and their travel limits.

Question 4 — Assembly validation

Question: Assemble the links and joints into a complete URDF robot model.

Answer: I assembled the link meshes, inertial data, joint relationships, limits, and control declarations in `robot_arm_urdf.urdf.xacro`. The fixed base joint anchors the structure at `world`; the five arm joints form a connected chain and the gripper joints branch from `link_5`.

I built the package with colcon, expanded the Xacro, and checked the resulting URDF. The parser reported `world` as the root and showed the expected arm chain and gripper branches.

```
cd /home/echo/moveit2_ws

colcon build --packages-select robot_arm_urdf

source install/setup.bash

xacro src/robot_arm_urdf/urdf/robot_arm_urdf.urdf.xacro > /tmp/robot_arm.urdf

check_urdf /tmp/robot_arm.urdf
```



Figure 5. Xacro source for the hardware declaration and arm joint interfaces.

Control source — gripper and simulator plugin

The ROS 2 control block also declares position commands and position/velocity feedback for both gripper joints.



Figure 6. Xacro source for gripper interfaces and the optional Gazebo Classic plugin.

Question 5 — RViz2 visualization

Question: Load the robot model in RViz and verify the visualization.

Answer: I launched the model using `display.launch.py`, which starts `robot_state_publisher`, the Joint State Publisher GUI, and RViz2. In RViz2, I set the Fixed Frame to `world` and enabled RobotModel. The screenshot shows the robot geometry and the RobotModel status as OK.

The Joint State Publisher GUI can be used to preview different joint positions. This demonstrates the model's kinematic visualization; it does not simulate forces or physical contact.

```
source /opt/ros/humble/setup.bash

source /home/echo/moveit2_ws/install/setup.bash

ros2 launch robot_arm_urdf display.launch.py
```

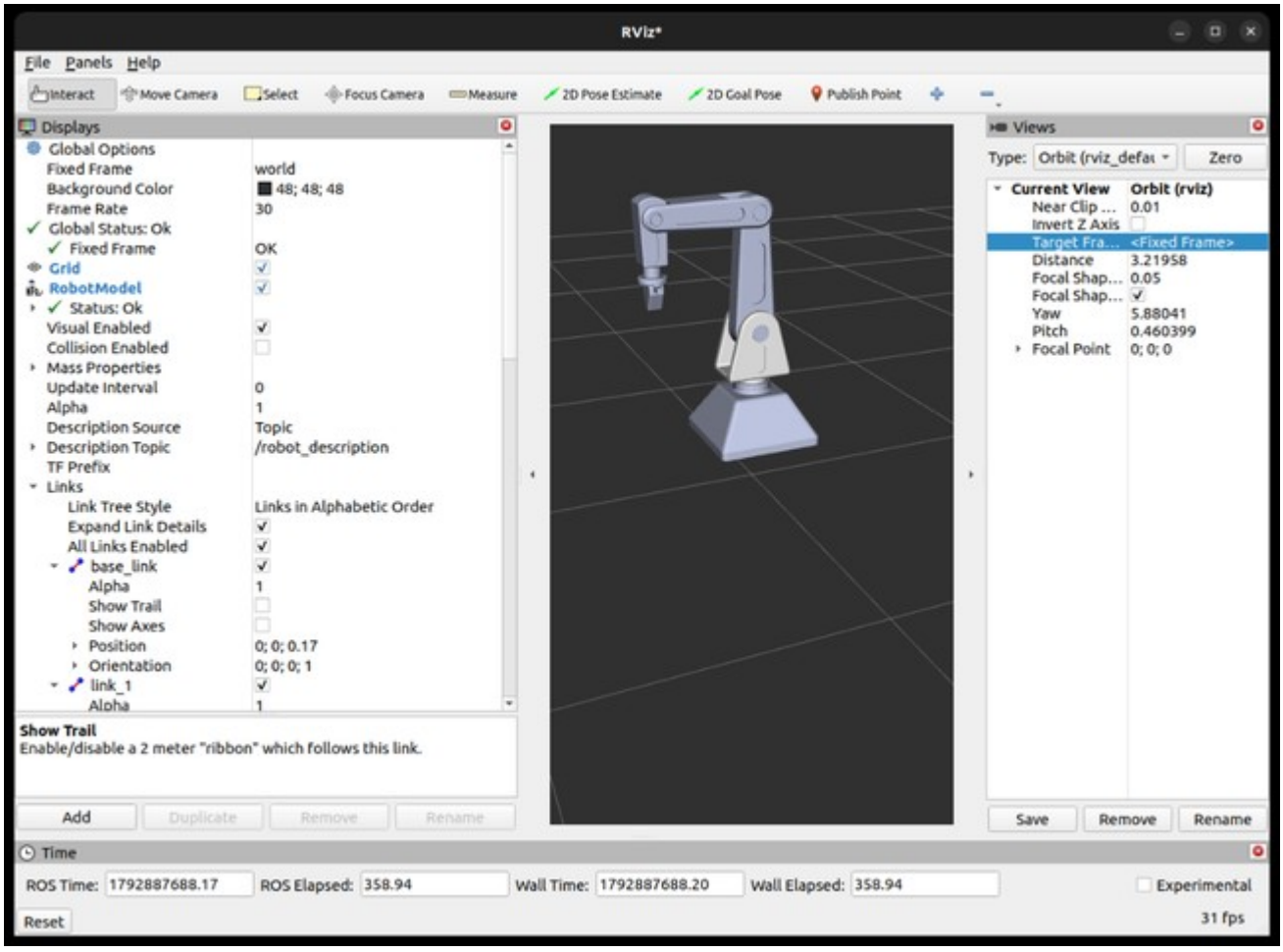


Figure 7. RViz2 visualization of the robot model with RobotModel status OK.

RViz2 launch source

This launch file evaluates the Xacro description, publishes the robot state, opens the joint slider GUI, and starts RViz2.

ROS 2 launch source — RViz2 visualization

--- display.launch.py ---

```python
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.substitutions import Command
from launch_ros.actions import Node
from launch_ros.parameter_descriptions import ParameterValue


def generate_launch_description():
    pkg = get_package_share_directory('robot_arm_urdf')
    xacro_file = os.path.join(pkg, 'urdf', 'robot_arm_urdf.urdf.xacro')
    robot_description = ParameterValue(Command(['xacro ', xacro_file]), value_type=str)

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{'robot_description': robot_description}],
        ),
        Node(
            package='joint_state_publisher_gui',
            executable='joint_state_publisher_gui',
        ),
        Node(
            package='rviz2',
            executable='rviz2',
        ),
    ])
```

Figure 8. Source of `display.launch.py`.

Question 6 — TF tree verification

Question: Visualize and verify the robot's TF tree using ROS 2 tools.

Answer: I generated the frame report using `tf2_tools view_frames`. It lists eight frames: `world`, `base_link`, and `link_1` through `link_7`. The tree shows the fixed base connection, the serial arm chain, and two gripper branches from `link_5`.

```
source /opt/ros/humble/setup.bash

source /home/echo/moveit2_ws/install/setup.bash

ros2 run tf2_tools view_frames
```

The generated TF report PDF and a readable image of the tree are included with the project files.



Figure 9. TF tree generated from the robot's published frames.

Controller configuration

I grouped the five revolute joints under `arm_controller` and the two prismatic gripper joints under `hand_controller`. The controller configuration requests position commands and position/velocity state interfaces. The included Gazebo control plugin is for Gazebo Classic; the RViz2 visualization and TF checks above are the demonstrations documented in this report.

ROS 2 controller configuration source

--- ros2_controllers.yaml ---

```yaml
controller_manager:
  ros__parameters:
    update_rate: 100

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    arm_controller:
      type: joint_trajectory_controller/JointTrajectoryController

    hand_controller:
      type: joint_trajectory_controller/JointTrajectoryController

arm_controller:
  ros__parameters:
    joints:
      - joint_1
      - joint_2
      - joint_3
      - joint_4
      - joint_5
    command_interfaces:
      - position
    state_interfaces:
      - position
      - velocity

hand_controller:
  ros__parameters:
    joints:
      - joint_6
      - joint_7
    command_interfaces:
      - position
    state_interfaces:
      - position
      - velocity
```

Figure 10. Controller configuration for the arm and gripper.

Submission checklist

1. This report PDF with answers to all six project questions.
2. The full `robot_arm_urdf/` package: Xacro, launch files, controller YAML, meshes, `package.xml`, and `CMakeLists.txt`.
3. The generated TF-tree PDF plus RViz2 and TF-tree screenshots.
4. The Xacro, launch, and controller source screenshots included in the `screenshots/` folder.

The editable report source is `report.html`. All submission materials are grouped under `ros2mini project/` for upload to the Drive folder.